Comparative Evaluation of Decision Trees and Ensemble Models for Network Intrusion Detection

24717274
Computer Science Division, Stellenbosch University
Stellenbosch, South Africa
Email: 24717274@sun.ac.za

***Abstract*—A network intrusion detection system (NIDS) classifies heterogeneous network flows across severely imbalanced traffic categories. High overall accuracy on skewed traffic often conceals poor recognition of rare cyberattacks. The present study investigated how a standalone decision tree compared with decision-tree ensemble methods on a ten-class subset of the University of New South Wales network-based 2015 (UNSW-NB15) dataset comprising 257,673 cleaned observations and 41 predictors. An extended iterative dichotomiser three (ID3) learner served as the common base classifier across all models. The controlled design isolated the effects of post-pruning, bootstrap aggregation, and stagewise additive modelling using a multi-class exponential loss function (SAMME) from differences between unrelated model families.**

The dataset was divided into a fixed stratified four-way split consisting of 123,683 growth rows, 41,225 reduced-error pruning rows, 41,234 validation rows, and 51,531 reserved generalisation rows. Preliminary learning-curve experiments up to 50,000 rows and a feature-correlation ablation showed that training sample size and minority class support strongly governed predictive performance. Removing strongly correlated predictors degraded validation macro-averaged F1-score.

A final comparison used all 123,683 growth rows and a shared hybrid class-balancing policy. On the reserved generalisation set, a single reduced-error-pruned decision tree achieved the strongest performance with a mean macro-averaged F1-score of 0.5993, a balanced accuracy of 63.02%, and an overall accuracy of 80.73%. Bagging with 25 unpruned members improved over the standalone unpruned tree from 0.5545 to 0.5747 in mean macro-averaged F1-score. Boosting with 25 depth-eight SAMME members achieved a mean macro-averaged F1-score of 0.5408. A supplementary validation extension showed no additional macro-averaged F1-score gain when expanding bagging from 25 to 50 members. The results demonstrated that combining hybrid class-balanced training with reduced-error post-pruning enabled a single decision tree to outperform the tested unpruned bagging and depth-bounded boosting ensembles.

***Index Terms*—Bagging, Class Imbalance, Decision Trees, Ensemble Learning, Multiclass Adaptive Boosting, Network Intrusion Detection, Reduced-Error Pruning.**

I. Introduction

Modern communication networks face a wide spectrum of cyberattacks ranging from high-volume denial-of-service floods and automated reconnaissance scans to low-footprint shellcode injections, backdoors, and self-propagating worms. A network intrusion detection system (NIDS) inspects flow-level packet, protocol, and content attributes to distinguish benign traffic from specific attack categories [4]. Framing multi-category intrusion detection as a supervised classification problem introduces several practical challenges. Network traffic combines high-cardinality categorical protocol descriptors with skewed continuous rate and timing counters. Flow statistics also exhibit strong pairwise collinearity and severe class imbalance where benign flows and common attacks outnumber rare exploits by orders of magnitude.

Decision trees provide a natural baseline for intrusion detection because tree induction accommodates mixed categorical and numerical predictors without requiring feature rescaling or one-hot encoding, and represents predictions as explicit rule paths [3], [5]. However, unrestricted decision trees grown greedily until leaf homogeneity exhibit high variance and memorise noisy or conflicting flow records. Two primary paradigms exist to address overfitting in tree-based classifiers. The first paradigm applies structural regularisation to a single tree through maximum depth constraints or post-pruning on held-out observations [3], [6]. The second paradigm constructs an ensemble that combines multiple base learners through bootstrap aggregation, known as bagging, or sequential adaptive reweighting, known as boosting [1], [2], [8].

The primary objective of the present study was to compare the predictive performance of a standalone decision tree with ensembles of decision trees for multiclass network intrusion detection on the University of New South Wales network-based 2015 (UNSW-NB15) dataset [4]. Rather than comparing an ensemble against a dissimilar model family such as nearest-neighbour or linear classifiers, the experimental design held the base learner family fixed across all experiments. Every standalone and ensemble model used the same custom iterative dichotomiser three (ID3) decision-tree implementation extended for continuous thresholds and reduced-error post-pruning. Holding the base learner constant ensured that observed differences in classification performance, class-level recall, and computational cost reflected the regularisation or aggregation mechanism rather than differences in feature representation.

To investigate the comparative objective systematically, the 257,673-row dataset was partitioned into fixed, stratified growth, pruning, validation, and reserved generalisation sets containing 123,683, 41,225, 41,234, and 51,531 rows, respectively. Four specific research questions guided the empirical progression:

1) How did predictive performance scale as the training sample size increased, and how strongly did natural class scarcity in small samples constrain minority-attack recognition?

2) How did maximum tree depth limits, reduced-error post-pruning, and class-balanced resampling interact to trade off overall accuracy against minority-class recall and precision?
3) How did bagging and stagewise additive modelling using a multi-class exponential loss function (SAMME) behave as ensemble size, base-learner depth, and learning rate varied?
4) Under a matched full-growth-pool budget and a shared hybrid class-balancing policy, how did an unpruned tree, a reduced-error-pruned tree, a bagging ensemble, and a SAMME boosting ensemble compare on the untouched generalisation set?

The empirical results showed that ensemble aggregation did not automatically outperform a single regularised decision tree when macro-averaged F1-score across all ten traffic categories served as the primary metric. In learning-curve experiments from 2,000 to 50,000 rows, increasing the volume of original training data yielded much larger macro-averaged F1-score gains than ensembling at small sample sizes, while unweighted accuracy-based pruning on naturally skewed samples reduced macro-averaged F1-score by collapsing sparse minority branches. In the final comparison using all 123,683 growth rows under a hybrid class-balancing policy and pruning against all 41,225 dedicated pruning rows, the effect of pruning reversed. Training-set balancing allowed minority-attack subtrees to grow during induction, while reduced-error pruning on the natural pruning distribution removed overfitted leaves with high false-positive rates. On the reserved generalisation set, the pruned single tree achieved the highest mean macro-averaged F1-score of 0.5993, the highest balanced accuracy of 63.02%, and the highest overall accuracy of 80.73%, outperforming 25-member bagging at 0.5747, the unpruned tree at 0.5545, and 25-member depth-eight SAMME boosting at 0.5408. A supplementary validation extension confirmed that expanding bagging from 25 to 50 members produced no additional gain in validation macro-averaged F1-score.

The remainder of the report is organised into five sections. Section II reviews the theoretical foundations of information-based decision trees, pruning strategies, and ensemble learning. Section III details the dataset preprocessing, the custom decision-tree and ensemble implementations, algorithmic optimisations, and the hybrid class-balancing formulation. Section IV describes the four-way partitioning protocol, evaluation metrics, and the sequential design of the experimental stages. Section V presents and analyses the empirical results across the development and generalisation sets. Finally, Section VI summarises the conclusions, methodological limitations, and directions for future work.

II. Background and Theoretical Foundations

Theoretical concepts underlying tree-based classification and ensemble aggregation are reviewed in Section II. Subsection II-A defines top-down decision-tree induction and Shannon entropy. Subsection II-B introduces alternative split criteria and binary threshold partitioning for ordered numerical attributes. Subsection II-C contrasts pre-pruning and reduced-error post-pruning. Finally, Subsection II-D formulates bootstrap aggregation and multiclass adaptive boosting.

A. Information-Based Decision Tree Learning

Decision tree algorithms induce a hierarchical, rule-based classifier by recursively partitioning a labelled dataset $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$, where n denotes the number of instances [3], [5]. Each instance $i \in \{1, \ldots, n\}$ is described by a vector $\mathbf{x}_i = (x_{i,1}, \ldots, x_{i,m})$ of m descriptive attributes and belongs to one of k mutually exclusive target classes in the class set $C = \{c_1, \ldots, c_k\}$. Internal nodes in a decision tree represent tests on individual descriptive attributes, branches correspond to test outcomes, and terminal leaf nodes assign a target class label from C. Multiple tree structures fit the same training instances, so top-down tree induction applies a preference bias toward shorter trees with fewer test nodes by placing the most discriminative attribute tests near the root [3].

In the standard ID3 algorithm introduced by Quinlan [5], tree induction proceeds top-down by greedily selecting the descriptive attribute $b \in \{1, \ldots, m\}$ that yields the greatest reduction in target class uncertainty. Uncertainty within a dataset D is quantified using the entropy measure introduced by Shannon [7], defined as

$$h(D) = -\sum_{j=1}^{k} \nu(D, c_j) \log_2 \nu(D, c_j), \tag{1}$$

where $h(D)$ denotes the Shannon entropy of D in bits, $j \in \{1, \ldots, k\}$ indexes the target classes, and $\nu(D, c_j)$ denotes the empirical proportion of instances in D with target label $y_i = c_j$, taking $0 \log_2 0 = 0$ by convention. A homogeneous dataset containing only a single target class has $h(D) = 0$, whereas a uniform class distribution over k classes attains the maximum entropy of $\log_2 k$ bits.

When a categorical attribute b with discrete domain L_b partitions D into disjoint subsets $D_l = \{(\mathbf{x}_i, y_i) \in D \mid x_{i,b} = l\}$ for each level $l \in L_b$, the expected remaining entropy $\xi(b, D)$ is the weighted sum of the subset entropies, given by

$$\xi(b, D) = \sum_{l \in L_b} \frac{|D_l|}{|D|} h(D_l), \tag{2}$$

where $|D_l|$ and $|D|$ denote the cardinalities of subset D_l and dataset D. The information gain $\iota(b, D)$ of partitioning D on attribute b is the resulting reduction in entropy, expressed as

$$\iota(b, D) = h(D) - \xi(b, D). \tag{3}$$

Recursive partitioning terminates along a branch under three stopping conditions [3]. First, if all reaching instances share the same target label, a leaf node is created with that target label. Second, if no candidate attributes remain, a leaf node is created with the modal target class of the reaching instances. Third, if subset D_l is empty, a leaf node is created with the modal target class of the parent partition D.

B. Alternative Impurity Metrics and Continuous Splits

Quinlan [5] showed that information gain favours categorical attributes with large domains L_b, because a high-cardinality attribute fragments D into numerous small, low-entropy subsets D_l even when the attribute carries weak generalisation value. To penalise multi-way fragmentation, Quinlan [5], [6] defined the gain ratio $\psi(b, D)$ by normalising information gain by the split information $\kappa(b, D)$ according to

$$\begin{aligned} \psi(b, D) &= \frac{\iota(b, D)}{\kappa(b, D)}, \\ \kappa(b, D) &= -\sum_{l \in L_b} \frac{|D_l|}{|D|} \log_2 \frac{|D_l|}{|D|}. \end{aligned} \tag{4}$$

In the C4.5 system, Quinlan [6] combined Equation (4) with an eligibility constraint that restricted selection to candidate splits whose information gain $\iota(b, D)$ was at least the mean information gain across all tested splits at that node.

An alternative impurity criterion is the Gini impurity $\zeta(D)$. The quantity $\zeta(D)$ measures the probability of misclassifying a randomly selected instance from D when labels are drawn according to the empirical class proportions $\nu(D, c_j)$ [3], defined as

$$\begin{aligned} \zeta(D) &= \sum_{j=1}^{k} \nu(D, c_j)\big(1 - \nu(D, c_j)\big) \\ &= 1 - \sum_{j=1}^{k} \nu(D, c_j)^2. \end{aligned} \tag{5}$$

Substituting $\zeta(\cdot)$ for $h(\cdot)$ in Equations (2) and (3) yields the Gini impurity reduction.

To incorporate ordered numerical attributes into an ID3-style learner, multi-way categorical branching is replaced by binary threshold tests of the form $x_{i,b} < \theta$ versus $x_{i,b} \geq \theta$, partitioning D into a left subset D_1 and a right subset D_2 [3], [6]. Sorting the distinct observed values $v_1 < v_2 < \cdots < v_u$ of attribute b restricts candidate thresholds θ to midpoints between adjacent values v_τ and $v_{\tau+1}$, where $\tau \in \{1, \ldots, u-1\}$ and u denotes the number of distinct values, whenever the associated target class labels differ between v_τ and $v_{\tau+1}$. The threshold θ that maximises impurity reduction between D_1 and D_2 is selected as the best split for attribute b. Unlike a categorical attribute removed after a multi-way split, a numerical attribute remains available to descendant nodes for further binary splitting at finer thresholds [3].

C. Overfitting and Tree Pruning

Unrestricted recursive partitioning continues until every leaf is homogeneous or all attributes are exhausted, causing deep subtrees to fit sampling noise and label conflicts in D [3]. As tree depth increases, leaf predictions rely on progressively smaller subsets of instances, increasing estimator variance on unseen data. Two structural regularisation families address tree overfitting:

- **Pre-pruning:** Tree induction halts early when a stopping condition is reached, such as a maximum edge depth d from the root or a minimum node cardinality. Pre-pruning reduces training computation directly, but introduces a horizon effect where locally weak splits that would enable informative descendant splits are prematurely blocked [3].
- **Post-pruning:** The decision tree is first grown to full depth on the fitting dataset D_f and then simplified bottom-up. In reduced-error pruning [3], [6], a separate pruning dataset P is routed through the induced tree in a post-order traversal. At each internal node v, the unweighted count of classification errors e_v committed by the existing subtree on the reaching pruning subset P_v is compared against the error count o_v committed if node v were replaced by a single leaf predicting the modal class y_v^* of the reaching fitting subset D_v. If $o_v \leq e_v$, the subtree rooted at v is replaced by the leaf y_v^*.

Because standard reduced-error pruning minimises unweighted classification error counts on P, the pruning criterion interacts with class imbalance: on skewed datasets, collapsing small minority-class subtrees reduces total errors on P and increases overall accuracy while reducing minority-class recall.

D. Ensemble Learning: Bagging and Multiclass Boosting

Ensemble learning combines the predictions of t base learners $\pi_1, \ldots, \pi_t$ to improve generalisation relative to an individual base classifier [3]. The two principal homogeneous ensemble families for decision trees are bootstrap aggregation and adaptive boosting.

1) Bootstrap Aggregation and Sub-Bagging: Bootstrap aggregation, introduced by Breiman [1] as bagging, trains t independent base learners on resampled datasets $D_1, \ldots, D_t$, where each D_r for $r \in \{1, \ldots, t\}$ is constructed by drawing $n = |D|$ instances uniformly with replacement from D. For classification, the ensemble prediction $\hat{\pi}(\mathbf{x}_i)$ aggregates hard class votes by uniform majority voting according to

$$\hat{\pi}(\mathbf{x}_i) = \arg\max_{c_j \in C} \sum_{r=1}^{t} \delta\big(\pi_r(\mathbf{x}_i) = c_j\big), \tag{6}$$

where $\delta(\cdot)$ denotes the indicator function returning 1 when the equality inside the argument holds and 0 otherwise. Because each uniform bootstrap sample omits a fraction $(1-1/n)^n \approx \exp(-1) \approx 0.368$ of the instances in D, unstable learners such as unpruned decision trees induce diverse tree structures across the t replicates. Breiman [1] showed that aggregating unstable predictors across bootstrap replicates reduces prediction variance caused by sample perturbations, while leaving the functional form of each individual base tree unchanged. Sub-bagging instead draws proper subsets of size $\omega < n$ without replacement, increasing sample diversity across the t learners while providing fewer training instances per tree. When bagging evaluates all m attributes at every split rather than sampling random attribute subspaces, the ensemble remains standard bagging rather than a random forest.

2) Multiclass Adaptive Boosting: Whereas bagging trains high-capacity base learners in parallel, adaptive boosting sequentially trains weak base learners so that each round $r \in \{1, \ldots, t\}$ emphasises training instances misclassified by earlier learners [2], [3]. Each training instance $i \in \{1, \ldots, n\}$ is assigned a non-negative weight $w_{r,i}$, initialised uniformly

at $w_{1,i} = 1/n$. At round r, a base learner π_r is induced using the weight vector $\mathbf{w}_r = (w_{r,1}, \ldots, w_{r,n})$, either via weighted impurity calculations or by drawing n instances with replacement according to $\mathbf{w}_r$, and is evaluated across all n instances of D to compute the weighted training error ϵ_r defined as

$$\epsilon_r = \frac{\sum_{i=1}^{n} w_{r,i}\delta\big(\pi_r(\mathbf{x}_i) \neq y_i\big)}{\sum_{i=1}^{n} w_{r,i}}. \tag{7}$$

For binary classification with $k = 2$, Freund and Schapire [2] required $\epsilon_r < 1/2$ to assign a positive learner weight. In a k-class classification problem with $k > 2$, however, random guessing achieves an accuracy of $1/k$, making the binary condition $\epsilon_r < 1/2$ overly restrictive for shallow trees. Zhu *et al.* [8] proposed the SAMME algorithm to generalise adaptive boosting to k classes by requiring only $\epsilon_r < 1 - 1/k$ and defining the learner voting weight α_r with learning rate $\eta > 0$ as

$$\alpha_r = \eta \left(\ln \frac{1 - \epsilon_r}{\epsilon_r} + \ln(k - 1) \right). \tag{8}$$

Observation weights are updated for round $r+1$ by multiplying the weights of misclassified instances by $\exp(\alpha_r)$ according to

$$w_{r+1,i} = \frac{w_{r,i} \exp\big(\alpha_r \delta(\pi_r(\mathbf{x}_i) \neq y_i)\big)}{z_r}, \tag{9}$$

where $z_r = \sum_{i=1}^{n} w_{r,i} \exp\big(\alpha_r \delta(\pi_r(\mathbf{x}_i) \neq y_i)\big)$ normalises the weight vector $\mathbf{w}_{r+1}$. The boosted ensemble predicts by confidence-weighted voting across the t learners according to

$$\hat{\pi}(\mathbf{x}_i) = \arg\max_{c_j \in C} \sum_{r=1}^{t} \alpha_r \delta\big(\pi_r(\mathbf{x}_i) = c_j\big). \tag{10}$$

Zhu *et al.* [8] proved that Equations (8)–(10) minimise a multiclass exponential loss via forward stagewise additive modelling and reduce to standard binary adaptive boosting when $k = 2$ and $\eta = 1$.

III. Methodology and Implementation

Data preparation and algorithmic implementations developed for the study are described in Section III. Subsection III-A details the preprocessing of the intrusion detection dataset and the representation of descriptive attributes. Subsection III-B explains the extended decision-tree learner and the incremental numerical threshold scorer. Subsection III-C describes the bottom-up reduced-error pruning implementation. Subsection III-D presents the generic bagging and multiclass boosting frameworks. Finally, Subsection III-E formulates the hybrid class-balanced resampling policy.

A. Dataset Preprocessing and Feature Representation

The empirical study used a subset of the UNSW-NB15 dataset [4] containing $n = 257{,}673$ network flow records. The target attribute `attack_cat` assigned each record to one of $k = 10$ traffic categories: *Normal*, *Reconnaissance*, *Backdoor*, *DoS*, *Exploits*, *Analysis*, *Fuzzers*, *Worms*, *Shellcode*, and *Generic*, indexed by $j \in \{1, \ldots, 10\}$ corresponding to integer codes zero through nine.

Exploratory data analysis guided four preprocessing steps prior to model induction:

1) **Removal of non-predictive and duplicate attributes:** The record identifier `id` was removed so that decision trees did not split on row indices. The binary attribute `is_ftp_login` was also removed because exploratory analysis showed that `is_ftp_login` duplicated the file transfer protocol (FTP) command counter `ct_ftp_cmd`.
2) **Encoding of informative missingness in `service`:** Missing values occurred only in the application-layer `service` attribute, and the rate of missingness varied substantially across attack categories. Specifically, `service` was missing in 100.0% of *Shellcode* records, 95.28% of *Backdoor* records, and 90.65% of *Fuzzers* records, compared with 1.55% of *Generic* records and 14.94% of *Worms* records. To retain the predictive signal associated with unidentified application services, missing entries in `service` were replaced with the explicit category string `'other'`.
3) **Separation of categorical and ordered numerical attributes:** After cleaning, $m = 41$ descriptive attributes remained. Four attributes were represented as discrete categorical predictors: transport protocol `proto` with 133 distinct levels in the growth pool, application service `service`, connection state `state`, and the binary address-port indicator `is_sm_ips_ports`. The remaining 37 attributes, comprising flow durations, packet inter-arrival times, bitrates, transmission control protocol (TCP) round-trip times, and integer packet, byte, and connection counters, were represented as ordered numerical predictors tested via binary threshold splits. Because decision trees test categorical levels directly and partition numerical attributes by order-preserving thresholds, neither numerical feature scaling nor one-hot encoding was applied.
4) **Retention of correlated predictors and repeated flow vectors:** Pairwise correlation screening across the numerical predictors in the growth pool identified 20 attribute pairs satisfying $\max(|\mu|, |\rho|) \geq 0.95$, where μ denotes the Pearson linear correlation coefficient and ρ denotes the Spearman rank correlation coefficient. Strong examples included `sbytes` and `sloss` with $\mu = 0.9958$, and `tcprtt` and `synack` with $\rho = 0.9963$. In greedy tree induction, two globally correlated counters provide distinct local split thresholds, and anomalous traffic often manifests on records where coupled network counters diverge. A preliminary complete-link correlation ablation on 2,000 fitting rows reduced the 41 predictors to 27 under two target-independent representative policies, lowering mean validation macro-averaged F1-score by 0.0151 and 0.0030 for unpruned trees and by 0.0218 and 0.0078 for pruned trees. Consequently, all 41 cleaned predictors were retained. Repeated identical 41-attribute vectors were likewise retained during preprocessing because flow multiplicity reflects natural traffic frequencies that inform both node entropy calculations and majority-

class selection during reduced-error pruning.

B. Extended Decision Tree Implementation

All decision-tree models were implemented in Python using `pandas` data frames with the target attribute placed in the final column. The base learner extended the categorical ID3 algorithm [5] to support continuous threshold splits, configurable impurity functions, optional maximum depth pre-pruning at depth d, and model persistence to disk via the `pickle` module.

1) Tree Structure and Unseen-Category Fallbacks: Induced decision trees were represented as nested Python dictionaries where terminal leaves stored scalar class labels. A categorical internal node mapped the selected attribute name to a dictionary of child subtrees keyed by each domain level $l \in L_b$. A numerical internal node mapped a two-element tuple of the attribute name and threshold θ to a dictionary containing the left branch for $x_{i,b} < \theta$ and the right branch for $x_{i,b} \geq \theta$. A categorical attribute selected at an internal node was removed from the candidate set along descendant branches, whereas a numerical attribute remained available for subsequent threshold splits deeper in the tree.

When trees were trained on small subsamples or resampled replicates, rare categorical levels present in the broader growth pool were absent from individual local training partitions. To ensure that every known category had a valid branch during pruning and evaluation, the learner accepted a domain dictionary mapping each categorical attribute b to the complete level set L_b observed in the growth pool. Any level $l \in L_b$ receiving zero instances at an internal node was assigned a leaf predicting the modal target class of that node. If an evaluation record contained a categorical value absent from the entire growth-pool domain L_b, the experiment prediction wrapper returned the global modal training class of the fitted model.

2) Incremental Numerical Threshold Scoring: Evaluating candidate numerical thresholds by constructing a new boolean mask and rescanning the data frame for every midpoint θ becomes computationally prohibitive when $n > 10^5$ across 37 numerical attributes. To make full-pool induction and ensemble training tractable, an incremental single-sort threshold scorer was implemented that produced exact mathematical equivalence to exhaustive partition scoring while reducing per-attribute complexity from $u \cdot n$ scans to $n \log_2 n + uk \log_2 k$ operations for n instances, u distinct attribute values, and k classes:

1) The node partition D was sorted once by the candidate numerical attribute b using a stable sort, and target labels were mapped to integer codes.
2) Identical values of attribute b were grouped into ordered pairs $(v_1, \mathbf{c}_1), \ldots, (v_u, \mathbf{c}_u)$, where v_τ for $\tau \in \{1, \ldots, u\}$ denotes the τ-th distinct value and $\mathbf{c}_\tau$ denotes the k-element class-count vector at value v_τ. Grouping equal values prior to threshold generation ensured that identical attribute values were never split across branches.
3) A right-branch class-count vector $\mathbf{c}_2$ was initialised to the total node class distribution and a left-branch class-count vector $\mathbf{c}_1$ was initialised to the zero vector. Scanning sequentially across adjacent value groups τ and $\tau + 1$, the scorer transferred $\mathbf{c}_\tau$ from $\mathbf{c}_2$ to $\mathbf{c}_1$ in k arithmetic operations.
4) Whenever two adjacent groups τ and $\tau + 1$ were both homogeneous and shared the same class label, the boundary between v_τ and $v_{\tau+1}$ did not improve impurity and was skipped [3]. For all other adjacent pairs, the midpoint $\theta_\tau = v_\tau/2 + v_{\tau+1}/2$ was scored directly from the updated count vectors $\mathbf{c}_1$ and $\mathbf{c}_2$.

C. Bottom-Up Reduced-Error Post-Pruning

Given an induced tree, the fitting dataset D_f, and a disjoint pruning dataset P, the reduced-error pruning routine returned a pruned copy without mutating the original unpruned tree. Consequently, each unpruned tree and the corresponding pruned tree shared the exact same initial fit.

The pruning algorithm traversed the tree in post-order, visiting all child subtrees before their parent node while routing the corresponding subsets $D_v \subseteq D_f$ and $P_v \subseteq P$ to each internal node v:

- At internal node v, the modal class y_v^* of the reaching fitting instances D_v was determined, resolving ties by the first modal value in series order. If D_v was empty, the subtree at v was left unchanged and scored once on P_v.
- Each recursively pruned child of v returned both the pruned child subtree structure and the integer count of unweighted classification errors committed on the routed child subset of P_v. Summing the child error counts yielded the retained subtree error count e_v without re-predicting descendant paths.
- The error count o_v of replacing the subtree at v with a single leaf predicting y_v^* was computed on the reaching pruning subset P_v as

$$o_v = \sum_{(\mathbf{x}_i, y_i) \in P_v} \delta(y_i \neq y_v^*). \tag{11}$$

- If $o_v \leq e_v$, the subtree at v was replaced by the leaf y_v^* and returned o_v to the parent node; otherwise, the pruned subtree was retained and returned e_v.

Because equality between o_v and e_v favoured replacement, any internal node reached by zero pruning observations was collapsed into the leaf y_v^*.

D. Ensemble Frameworks

The ensemble module used a generic callback interface where each ensemble constructor accepted a model-training function and a single-row prediction function, returning a serialisable voting ensemble object.

1) Bagging and Sub-Bagging: For an ensemble of t base learners, standard bagging drew n instances with replacement per member $r \in \{1, \ldots, t\}$, whereas sub-bagging drew $\omega < n$ instances without replacement. Each base tree received a unit voting weight $\alpha_r = 1.0$. At inference time, hard predictions were combined via Equation (6), with voting ties broken deterministically by the order of first appearance of class labels in the training set. When evaluating ensemble prefixes across size checkpoints $t \in \{5, 10, 25, 35, 50\}$, member predictions on fixed evaluation partitions were cached so that each checkpoint evaluated only the newly added base trees.

2) Multiclass Adaptive Boosting: The boosting module implemented the multiclass SAMME algorithm [8] through stochastic weighted resampling so that the ID3 base learner did not require a weighted-impurity interface. At round $r \in \{1, \ldots, t\}$, n training indices were drawn with replacement according to the current weight vector $\mathbf{w}_r$. The induced base tree π_r was then evaluated across all n rows of the original fitting population to compute the weighted training error ϵ_r via Equation (7).

Two numerical and termination rules were enforced during boosting:

1) **Termination on zero or excessive error:** If $\epsilon_r = 0$ on the fitting population, boosting terminated immediately and retained π_r alone with weight $\alpha_1 = 1.0$. Conversely, if $\epsilon_r \geq 1 - 1/k$, corresponding to 0.9 for $k = 10$ classes, learner π_r was rejected and boosting terminated, returning the previously accepted learners.
2) **Log-space weight maintenance:** To prevent floating-point underflow in ϵ_r or overflow in $\exp(\alpha_r)$ across sequential rounds, instance weights were stored as normalised log-weights $\ell_{r,i} = \ln w_{r,i}$. Letting $W_r = \{i \in \{1, \ldots, n\} \mid \pi_r(\mathbf{x}_i) \neq y_i\}$ denote the set of misclassified training indices at round r and $\hat{\ell}_r = \max_{i \in W_r} \ell_{r,i}$, the log-error $\ln \epsilon_r$ was computed via the log-sum-exp identity as

$$\ln \epsilon_r = \hat{\ell}_r + \ln \sum_{i \in W_r} \exp(\ell_{r,i} - \hat{\ell}_r) - \ln \sum_{i=1}^{n} w_{r,i}. \quad (12)$$

After computing α_r via Equation (8), misclassified log-weights were incremented by α_r and re-normalised in log-space.

E. Hybrid Class-Balanced Training Resampling

The natural class distribution of the UNSW-NB15 growth pool is heavily skewed. The three most frequent categories, *Normal* at 36.09%, *Generic* at 22.85%, and *Exploits* at 17.28%, account for 76.22% of all growth records. Conversely, the four rarest categories, *Worms* at 0.07%, *Shellcode* at 0.59%, *Backdoor* at 0.90%, and *Analysis* at 1.04%, together constitute only 2.60% of the pool. Training on natural proportions provides very few minority instances in subsamples, whereas pure equal-quota balancing at 10% per class discards the majority of abundant-class records within a fixed draw budget n and heavily duplicates rare records.

To control class-balancing strength continuously while keeping the total number of training draws fixed at n, a hybrid sampling proportion γ_j was defined for each class c_j across $j \in \{1, \ldots, k\}$ using an interpolation weight $\lambda \in [0, 1]$ according to

$$\gamma_j = (1 - \lambda)\beta_j + \frac{\lambda}{k}, \quad (13)$$

where β_j denotes the natural proportion of class c_j in the growth pool. Setting $\lambda = 0$ preserved the natural class proportions β_j, setting $\lambda = 1$ enforced uniform equal-class quotas $1/k = 0.10$, and setting $\lambda = 0.5$ averaged the natural distribution with a uniform prior according to $\gamma_j = 0.5\beta_j + 0.05$. Integer class draw quotas n_j summing to n were computed by taking $\lfloor n\gamma_j \rfloor$ and assigning remaining draws in descending order of fractional remainders.

Within each class c_j, sampling adapted to the available source count in the growth pool:

- When the requested quota n_j did not exceed the available class count in the source pool, n_j records were drawn without replacement to maximise distinct flow coverage.
- When the requested quota n_j exceeded the available class count in the source pool, n_j records were drawn with replacement.

For a given random seed, the standalone decision tree and the SAMME boosting ensemble shared the same initial λ-balanced training sample of n draws, and SAMME subsequently performed weighted resampling on that sample. For balanced bagging, however, bootstrapping a single pre-balanced sample would permanently exclude unsampled majority-class records across all t members. Instead, each bagging member $r \in \{1, \ldots, t\}$ drew a fresh, independent λ-balanced sample of n draws directly from the full source growth pool using a deterministic member seed. Class balancing was applied exclusively to training samples; the pruning, validation, and generalisation partitions always retained their natural class proportions.

IV. Empirical Procedure

The experimental protocol used to evaluate standalone decision trees and decision-tree ensembles is defined in Section IV. Subsection IV-A describes the four-way stratified dataset partitioning and class quotas. Subsection IV-B defines the classification performance measures and evaluation scopes. Finally, Subsection IV-C specifies the four sequential experimental stages and their control parameter settings.

A. Stratified Four-Way Data Partitioning

To evaluate tree induction, reduced-error post-pruning, hyperparameter tuning, and final generalisation without data leakage, the cleaned 257,673-row dataset was partitioned once into four disjoint subsets via stratified random sampling within each target class using split seed 2026. First, 20% of each class was reserved for the generalisation set. The remaining 80% of each class was divided in a 60% : 20% : 20% ratio into growth, pruning, and validation pools:

- **Growth pool:** The growth pool contained 123,683 rows, representing 48.0% of the dataset, and served as the sole source of training observations for inducing standalone trees and ensemble members.
- **Pruning pool:** The pruning pool contained 41,225 rows, representing 16.0% of the dataset, and was used exclusively by bottom-up reduced-error pruning to evaluate whether internal subtrees should be replaced by modal-class leaves.
- **Validation pool:** The validation pool contained 41,234 rows, representing 16.0% of the dataset, and was used for all development comparisons, learning-curve and depth diagnostics, balancing-policy selection, and ensemble hyperparameter tuning. Separating the validation pool

from the pruning pool prevented optimistic bias, because reduced-error pruning uses pruning-set labels to modify tree structure.

- **Generalisation pool:** The generalisation pool contained 51,531 rows, representing 20.0% of the dataset, and was held in reserve throughout development and evaluated once after all primary model configurations were locked.

Table I lists the exact class counts across the four partitions alongside the natural class proportions β_j and the hybrid balanced quotas n_j for $\lambda = 0.5$ at $n = 123{,}683$ draws. Under $\lambda = 0.5$, the three abundant categories *Normal*, *Exploits*, and *Generic* were sampled without replacement, whereas the remaining seven categories were oversampled with replacement.

B. Performance Metrics and Evaluation Scopes

Because natural network traffic is dominated by *Normal*, *Generic*, and *Exploits* records, a trivial classifier predicting *Normal* for every observation achieves an overall accuracy of 36.09% on the validation set while detecting zero attacks. Every fitted model was therefore evaluated using four complementary metrics derived from the $k \times k$ confusion matrix $\mathbf{M}$, where scalar entry m_{jq} for $j, q \in \{1, \ldots, k\}$ denotes the number of observations of true class c_j predicted as class c_q:

1) **Per-class precision, recall, and F1-score:** For each target class c_j, precision p_j, recall s_j, and F1-score f_j are defined as

$$p_j = \frac{m_{jj}}{\sum_{q=1}^{k} m_{qj}}, \quad s_j = \frac{m_{jj}}{\sum_{q=1}^{k} m_{jq}}, \quad f_j = \frac{2 p_j s_j}{p_j + s_j}, \tag{14}$$

where $p_j = 0$ and $f_j = 0$ whenever the denominator of p_j or f_j is zero. Recall s_j measures detection sensitivity for category c_j, while precision p_j measures resistance to false alarms.

2) **Macro-averaged F1-score:** The primary development and ranking metric $\bar{f}$ is the unweighted mean of the k per-class F1-scores, defined as

$$\bar{f} = \frac{1}{k} \sum_{j=1}^{k} f_j. \tag{15}$$

The metric $\bar{f}$ weights rare attack classes equally with abundant classes and penalises models that increase minority recall by generating excessive false positives.

3) **Balanced accuracy:** The unweighted mean of per-class recalls $\bar{s}$ is defined as

$$\bar{s} = \frac{1}{k} \sum_{j=1}^{k} s_j. \tag{16}$$

4) **Overall accuracy:** The natural-distribution proportion of correctly classified observations a over an evaluation set of size n is defined as

$$a = \frac{1}{n} \sum_{j=1}^{k} m_{jj}. \tag{17}$$

When models were trained under class-balanced resampling with $\lambda > 0$, scoring a model on the unresampled natural growth pool included majority-class rows omitted from the balanced training draw. Two training-side evaluation scopes were therefore recorded: fit-population metrics, measured on the actual balanced fitting draws, and natural-growth metrics, measured on the unresampled source growth rows. All validation and generalisation metrics were evaluated on the full natural-distribution validation and generalisation partitions. Across repeated random seeds, sample means and sample standard deviations were computed. Because all seeds shared the same fixed validation and generalisation partitions, the across-seed standard deviation quantified sensitivity to training subsample and ensemble resampling variation rather than variance across independent test sets.

C. Progressive Experimental Stages

Four sequential experimental stages were executed. Every decision tree across all four stages was induced using information gain $\iota(b, D)$ and all $m = 41$ cleaned predictors.

1) Stage One: Sample-Size Learning Curves and Feature-Conflict Analysis: To measure how training sample volume governed single-tree generalisation and rare-class coverage, unrestricted ID3 trees were trained on stratified natural-proportion subsamples with $\lambda = 0$ of sizes $n \in \{2{,}000, 5{,}000, 10{,}000, 20{,}000, 50{,}000\}$ across three sampling seeds, 42, 43, and 44. Each induced tree was also post-pruned using a fixed 10,000-row stratified subsample of the pruning pool. In addition, each training subsample was grouped by exact 41-attribute vectors to determine the deterministic training-accuracy ceiling imposed by identical predictor vectors carrying conflicting target labels.

2) Stage Two: Tree Depth Sweeps and Equal-Quota Balancing at 10,000 Rows: At a common source budget of $n = 10{,}000$ growth rows across seeds 42, 43, and 44, two regularisation and balancing studies were conducted:

- **Maximum depth sweep:** Unpruned single trees were induced at maximum depth limits of $d \in \{0, 1, 2, 4, 8, 12, 20\}$ and at unrestricted depth under both natural proportions with $\lambda = 0$ and equal-quota balancing with $\lambda = 1.0$, where 1,000 draws were allocated per class, to examine underfitting and overfitting as tree capacity increased.
- **Matched 10,000-row method and policy comparison:** Under both $\lambda = 0$ and $\lambda = 1.0$, the unpruned tree and the reduced-error-pruned tree were compared against bagging with $t = 25$ unrestricted trees and SAMME boosting with $t = 25$ depth-three trees at $d = 3$ and learning rates $\eta \in \{0.5, 1.0\}$.

3) Stage Three: Full-Growth-Pool Hyperparameter Pilot: Because Stage One showed that validation macro-averaged F1-score had not plateaued at 50,000 rows and Stage Two showed that pure equal-quota balancing at $\lambda = 1.0$ reduced overall accuracy on *Normal* traffic, Stage Three scaled training to the full growth pool of $n = 123{,}683$ draws on seed 42 for a 12-job tuning pilot. The pilot crossed two hybrid balancing weights, $\lambda \in \{0.5, 1.0\}$, against:

TABLE I
STRATIFIED FOUR-WAY DATASET PARTITIONING ACROSS THE TEN TRAFFIC CATEGORIES AND HYBRID BALANCED QUOTAS

Index (j)	Category (c_j)	Growth	Pruning	Validation	Generalisation	Total	Share (β_j)	Balanced Quota (n_j)
1	Normal	44,640	14,880	14,880	18,600	93,000	36.09%	28,504, without replacement
2	Reconnaissance	6,714	2,238	2,238	2,797	13,987	5.43%	9,541, with replacement
3	Backdoor	1,118	372	374	465	2,329	0.90%	6,743, with replacement
4	DoS	7,849	2,616	2,618	3,270	16,353	6.35%	10,109, with replacement
5	Exploits	21,372	7,124	7,124	8,905	44,525	17.28%	16,870, without replacement
6	Analysis	1,285	428	429	535	2,677	1.04%	6,827, with replacement
7	Fuzzers	11,638	3,879	3,880	4,849	24,246	9.41%	12,003, with replacement
8	Worms	84	28	28	34	174	0.07%	6,226, with replacement
9	Shellcode	725	241	243	302	1,511	0.59%	6,547, with replacement
10	Generic	28,258	9,419	9,420	11,774	58,871	22.85%	20,313, without replacement
Total	**All Categories**	**123,683**	**41,225**	**41,234**	**51,531**	**257,673**	**100.00%**	**123,683 draws**

- An unrestricted single tree and the corresponding reduced-error-pruned tree, pruned using all 41,225 rows of the pruning pool.
- A five-member bagging ensemble with $t = 5$ unrestricted base trees.
- A five-round SAMME boosting ensemble with $t = 5$ across a grid of maximum base-tree depths $d \in \{3, 8\}$ and learning rates $\eta \in \{0.5, 1.0\}$.

Based on validation macro-averaged F1-score and overall accuracy in the pilot, the shared balancing parameter was locked at $\lambda = 0.5$ and the SAMME boosting configuration was locked at $d = 8$ and $\eta = 1.0$.

4) Stage Four: Locked Final Comparison and Bagging Extension: The primary final experiment compared four locked approaches on the full 123,683-row growth pool under the shared $\lambda = 0.5$ hybrid balancing policy across two matched training seeds, 42 and 43:

1) **Balanced unpruned tree:** A single unrestricted decision tree.
2) **Balanced pruned tree:** The same fitted unrestricted tree, post-pruned via reduced-error pruning using all 41,225 rows of the pruning partition.
3) **Balanced bagging ensemble:** An ensemble of $t = 25$ unrestricted, unpruned trees, each trained on a fresh $\lambda = 0.5$ sample of 123,683 draws from the growth pool using non-overlapping deterministic member seeds from 42 through 66 for seed 42 and from 43,000,000 through 43,000,024 for seed 43, combined by uniform majority voting.
4) **Balanced boosting ensemble:** Up to $t = 25$ sequential rounds of SAMME boosting with maximum base-tree depth $d = 8$ and learning rate $\eta = 1.0$, trained on the shared $\lambda = 0.5$ initial sample.

Ensemble validation checkpoints were recorded at $t \in \{5, 10, 25\}$. Training tasks were distributed across two Linux machines using eight worker processes on a Lenovo ThinkBook and six lower-priority worker processes on a Dell Vostro, with single-threaded numerical libraries and secure hash algorithm 256-bit (SHA-256) checksum verification of all frozen data and model files. After all 54 planned tasks completed, the eight locked primary models at the $t = 25$ ensemble endpoint were evaluated once on the reserved 51,531-row generalisation partition. Finally, an exploratory validation-only follow-up extended both bagging trajectories from $t = 25$ to $t = 35$ and $t = 50$ members without accessing the generalisation partition.

V. RESULTS AND DISCUSSION

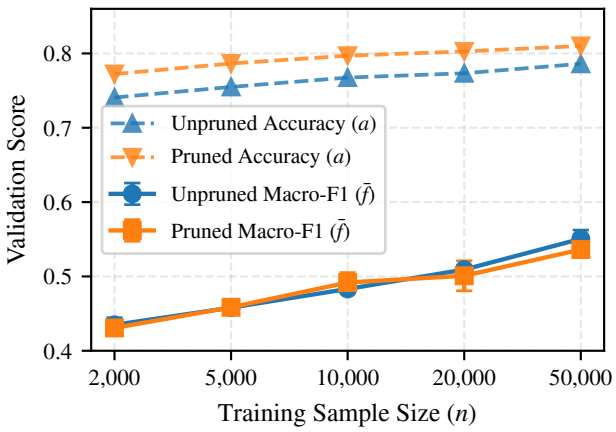
Empirical results across the four experimental stages defined in Section IV are presented and analysed in Section V. Subsection V-A examines the Stage One sample-size learning curves, the effect of reduced-error pruning on natural class distributions, and the deterministic training-accuracy ceiling imposed by conflicting predictor vectors. Subsection V-B analyses the Stage Two maximum-depth sweeps and the 10,000-row comparison between natural and equal-quota balanced policies. Subsection V-C reports the Stage Three full-growth-pool tuning pilot. Subsection V-D presents the Stage Four validation trajectories and the primary evaluation on the reserved generalisation set. Subsection V-E provides a class-level error analysis comparing the four locked approaches. Finally, Subsection V-F discusses the methodological limitations of the study.

A. Stage One: Sample-Size Scaling and Feature-Conflict Bounds

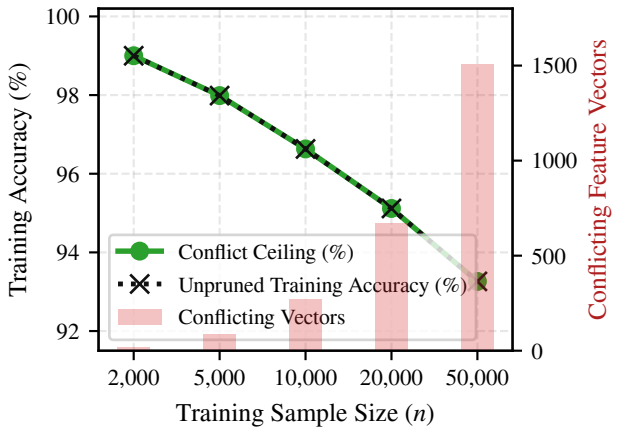
Table II and Figure 1 present the Stage One learning-curve results as the natural-proportion training sample size n at $\lambda = 0$ increased from 2,000 to 50,000 rows across three seeds. Figure 1a shows validation macro-averaged F1-score $\bar{f}$ and overall accuracy a, while Figure 1b compares observed in-sample training accuracy against the deterministic conflict ceiling.

Three primary findings emerged from Stage One:

1) **Sample size governed minority-attack recognition:** As shown in Table II and Figure 1a, unpruned validation macro-averaged F1-score increased monotonically with training sample size from 0.4349 at 2,000 rows to 0.5506 at 50,000 rows, representing a gain of 0.1157. Rare-class representation drove the observed gain: a 2,000-row proportional sample contained only one *Worms* observation, six *Shellcode* observations, and 18 *Backdoor* observations, whereas a 50,000-row sample contained 34 *Worms* observations and raised unpruned *Worms*



(a) Validation macro-averaged F1-score and overall accuracy.



(b) In-sample training accuracy and deterministic conflict ceiling.

Fig. 1. Stage One sample-size learning curves across $n \in \{2{,}000, 5{,}000, 10{,}000, 20{,}000, 50{,}000\}$ natural-proportion training rows across three seeds.

TABLE II
STAGE ONE LEARNING-CURVE RESULTS ON THE VALIDATION SET ACROSS THREE SEEDS

	Validation Set			Training Set
Rows (n) Tree	$\bar{f}$	a (%)	$\bar{s}$ (%)	a (%)
2,000 Unpruned	0.4349 ± 0.0095	74.07 ± 0.11	43.69 ± 1.28	99.00 ± 0.18
2,000 Pruned	0.4307 ± 0.0043	77.23 ± 0.16	43.15 ± 0.75	86.65 ± 0.93
5,000 Unpruned	0.4580 ± 0.0093	75.50 ± 0.42	46.14 ± 0.96	97.99 ± 0.14
5,000 Pruned	0.4585 ± 0.0068	78.65 ± 0.34	45.06 ± 1.23	85.94 ± 0.22
10,000 Unpruned	0.4832 ± 0.0021	76.75 ± 0.30	49.00 ± 0.45	96.63 ± 0.14
10,000 Pruned	0.4918 ± 0.0133	79.69 ± 0.13	48.38 ± 1.11	85.58 ± 0.30
20,000 Unpruned	0.5092 ± 0.0041	77.33 ± 0.18	51.17 ± 0.53	95.12 ± 0.03
20,000 Pruned	0.5009 ± 0.0203	80.27 ± 0.19	49.31 ± 1.70	85.62 ± 0.15
50,000 Unpruned	0.5506 ± 0.0118	78.61 ± 0.20	54.17 ± 1.31	93.26 ± 0.02
50,000 Pruned	0.5362 ± 0.0061	81.00 ± 0.21	51.21 ± 0.90	84.97 ± 0.22

validation F1-score to 0.5450. Because validation macro-averaged F1-score had not plateaued at 50,000 rows, subsequent final experiments used the entire 123,683-row growth pool.

2) **Divergence between accuracy and macro-averaged F1-score under natural pruning:** Reduced-error post-pruning increased validation overall accuracy at every tested sample size by 2.39 to 3.16 percentage points, reaching 81.00% at 50,000 rows while reducing the mean leaf count at 50,000 rows from 25,196 to 1,073. However, on naturally skewed samples, post-pruning reduced validation balanced accuracy at every sample size and lowered validation macro-averaged F1-score at 2,000, 20,000, and 50,000 rows, dropping from 0.5506 to 0.5362 at 50,000 rows. At 50,000 rows, pruning reduced mean validation recall on *Worms* from 48.8% to 22.6% and reduced *Worms* F1-score from 0.5450 to 0.3440. Minimising unweighted error counts on a naturally skewed pruning sample therefore penalised minority-class subtrees when training classes were unbalanced.
3) **Deterministic feature-conflict ceiling on training accuracy:** As shown in Table II, the in-sample training accuracy of unrestricted trees decreased from 99.00% at 2,000 rows to 93.26% at 50,000 rows. Grouping each training sample by all 41 predictor values in Figure 1b showed that the number of distinct 41-attribute vectors associated with multiple conflicting target labels grew from a mean of 19.0 at 2,000 rows to 1,508.0 at 50,000 rows out of 35,877 distinct predictor vectors. Because a deterministic classifier assigns a single label to each unique attribute vector, the maximum achievable training accuracy is bounded by predicting the modal label of each identical-vector group. Across all 15 unpruned fits in Table II, the induced ID3 trees matched the exact deterministic accuracy ceiling in every run, achieving 98.85%, 99.20%, and 98.95% at 2,000 rows, and 93.284%, 93.248%, and 93.254% at 50,000 rows.

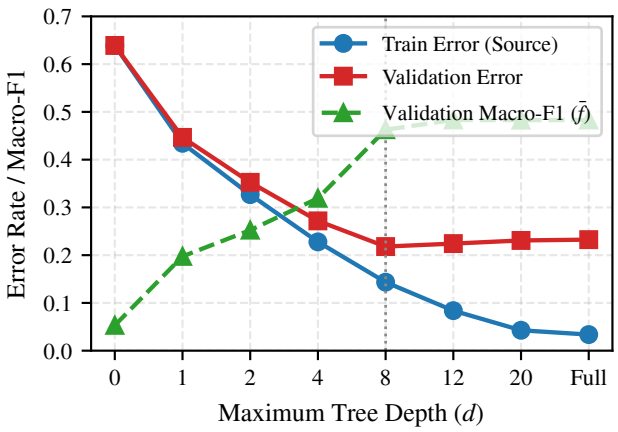
B. Stage Two: Tree Depth Regularisation and Equal-Quota Balancing at 10,000 Rows

Table III and Figure 2 report the Stage Two maximum-depth sweep across $d \in \{0, 1, 2, 4, 8, 12, 20\}$ and unrestricted growth at $n = 10{,}000$ rows under natural ($\lambda = 0$) and equal-quota balanced ($\lambda = 1.0$) policies.

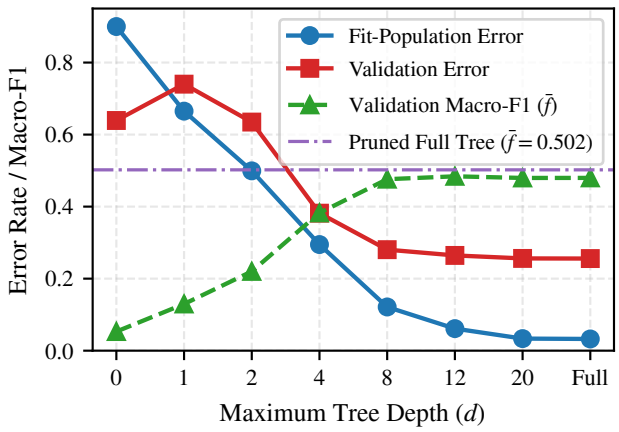
TABLE III
STAGE TWO MAXIMUM-DEPTH SWEEP AT 10,000 ROWS ACROSS THREE SEEDS

	Natural Policy ($\lambda = 0$)			Balanced Policy ($\lambda = 1.0$)		
Depth (d)	Train	Validation	$\bar{f}$	Fit	Validation	$\bar{f}$
0	0.6391	0.6391	0.0530	0.9000	0.6391	0.0530
1	0.4343	0.4466	0.1974	0.6651	0.7400	0.1296
2	0.3267	0.3525	0.2524	0.4993	0.6348	0.2201
4	0.2278	0.2718	0.3191	0.2947	0.3818	0.3825
8	0.1434	**0.2180**	0.4627	0.1213	0.2805	0.4761
12	0.0838	0.2242	0.4831	0.0613	0.2645	**0.4841**
20	0.0427	0.2308	0.4827	0.0335	0.2562	0.4796
Full	**0.0337**	0.2325	**0.4832**	**0.0325**	**0.2558**	0.4797
Pruned	0.1442	**0.2031**	**0.4918**	0.1482	**0.2366**	**0.5021**

Under the natural training policy shown in Table III and Figure 2a, source training classification error decreased mono-



(a) Natural training policy ($\lambda = 0$).



(b) Equal-quota balanced training policy ($\lambda = 1.0$).

Fig. 2. Stage Two maximum-depth regularisation curves at $n = 10{,}000$ rows across three seeds.

tonically from 0.6391 at $d = 0$ to 0.0337 at unrestricted depth, whereas validation classification error reached a minimum of 0.2180 at $d = 8$ and increased to 0.2325 at unrestricted depth. The divergence between training and validation error beyond $d = 8$ confirmed overfitting in deep unpruned trees under classification error. Validation macro-averaged F1-score $\bar{f}$ increased rapidly up to $\bar{f} = 0.4627$ at $d = 8$ and reached $\bar{f} = 0.4831$ at $d = 12$. Under equal-quota balancing in Figure 2b, fit-population error decreased from 0.9000 at $d = 0$ to 0.0325 at full depth, while validation $\bar{f}$ peaked at $\bar{f} = 0.4841$ for $d = 12$ and declined slightly to 0.4797 at full depth. Under both policies, bottom-up reduced-error post-pruning outperformed every fixed depth limit, reaching $\bar{f} = 0.4918$ under $\lambda = 0$ and $\bar{f} = 0.5021$ under $\lambda = 1.0$.

Table IV and Figure 3 compare the standalone trees against 25-member bagging and 25-round depth-three SAMME boosting at the matched 10,000-row budget:

- **Bagging improved over unpruned single trees:** Under natural proportions at $\lambda = 0$, 25-member bagging increased validation $\bar{f}$ relative to the unpruned tree in all three seeds by $+0.0162$, $+0.0208$, and $+0.0075$, raising mean $\bar{f}$ from 0.4832 to 0.4980. Under equal-quota balancing at $\lambda = 1.0$, 25-member bagging increased validation $\bar{f}$ over the balanced unpruned tree by $+0.0207$, $+0.0216$, and $+0.0230$, raising mean $\bar{f}$ from 0.4797 to 0.5015. Drawing a fresh balanced sample from the 10,000-row source pool for each bagging member also increased distinct source coverage from 4,831–4,846 rows in a single balanced tree to 9,999–10,000 rows across 25 members.
- **Interaction between class balancing and post-pruning:** Applying equal-quota balancing at $\lambda = 1.0$ to an unpruned single tree lowered mean validation $\bar{f}$ from 0.4832 to 0.4797, but post-pruning the balanced tree raised validation $\bar{f}$ in every seed to a mean of 0.5021 ± 0.0017. The balanced pruned single tree matched 25-member balanced bagging at $\bar{f} = 0.5015 \pm 0.0035$ while requiring a mean fitting time of 49.5 s compared with 1,094.9 s for balanced bagging.
- **Accuracy cost of pure equal-quota balancing and underfitting of depth-three boosting:** As shown in Figure 3b, equal-quota balancing at $\lambda = 1.0$ increased balanced accuracy $\bar{s}$ across all methods, but reduced overall accuracy a from 79.69% to 76.34% for the pruned tree and from 78.97% to 76.77% for bagging as validation recall on *Normal* traffic dropped from 92.3% to 82.9%. Furthermore, although depth-three SAMME boosting improved over a single depth-three tree from 0.2839 to 0.4383 under $\lambda = 0$, depth-three base learners lagged behind unrestricted trees and bagging across all settings. Under $\lambda = 1.0$ and $\eta = 0.5$, depth-three boosting achieved a balanced accuracy of 58.34% but an overall accuracy of only 67.34%, predicting *Worms* 775 to 1,239 times on a validation set containing 28 true *Worms* records, corresponding to 2.6% precision.

C. Stage Three: Full-Growth-Pool Hyperparameter Pilot

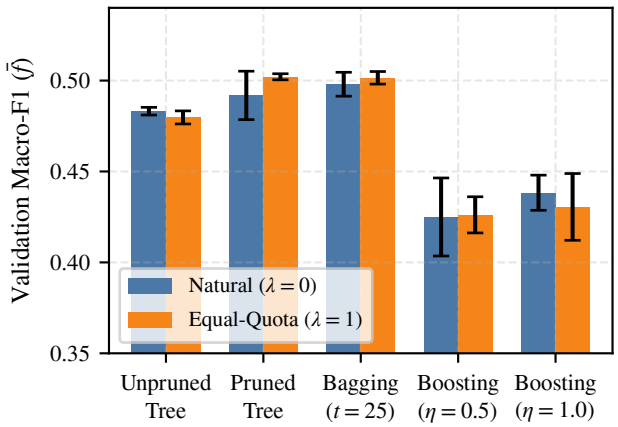
Stage Three evaluated the 12 pilot jobs on the full 123,683-row growth pool on seed 42 to compare $\lambda = 0.5$ against $\lambda = 1.0$ and to compare base-tree depths $d \in \{3, 8\}$ and learning rates $\eta \in \{0.5, 1.0\}$ for SAMME boosting. Table V summarises the resulting validation metrics and wall-clock runtimes.

Two findings from Table V determined the locked settings for Stage Four:

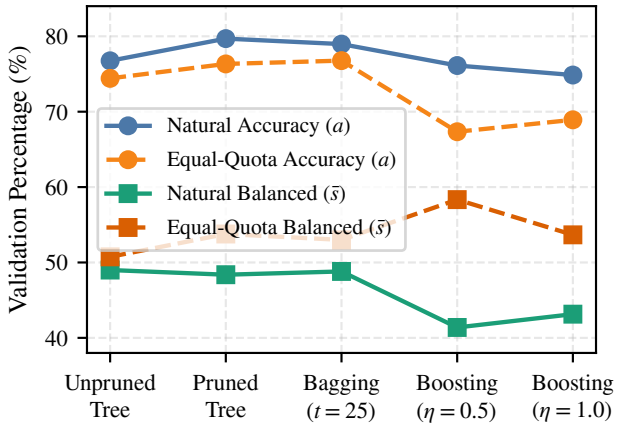
1) **Superiority of $\lambda = 0.5$ over $\lambda = 1.0$:** Across every model family in Table V, $\lambda = 0.5$ achieved higher validation macro-averaged F1-score $\bar{f}$ than $\lambda = 1.0$: 0.5673 versus 0.5603 for the unpruned tree, 0.6003 versus 0.5915 for the pruned tree, 0.5852 versus 0.5789 for five-member bagging, and 0.5587 versus 0.5374 for depth-eight boosting at $\eta = 1.0$. Simultaneously, $\lambda = 0.5$ preserved 1.87 to 3.14 percentage points higher validation overall accuracy a than $\lambda = 1.0$. Under $\lambda = 0.5$, each 123,683-draw sample retained 87,160 distinct source rows from the growth pool compared with 55,872 distinct source rows under $\lambda = 1.0$, preserving substantially

TABLE IV
STAGE TWO MATCHED COMPARISON AT 10,000 SOURCE GROWTH ROWS ACROSS NATURAL ($\lambda = 0$) AND EQUAL-QUOTA ($\lambda = 1.0$) POLICIES ACROSS THREE SEEDS

Method	Policy	Validation $\bar{f}$	Validation a (%)	Validation $\bar{s}$ (%)	Train Error (%)	Fit Time (s)
Unpruned Tree	Natural ($\lambda = 0$)	0.4832 ± 0.0021	76.75 ± 0.30	49.00 ± 0.45	3.37 ± 0.14	70.3 ± 4.5
Pruned Tree	Natural ($\lambda = 0$)	0.4918 ± 0.0133	$\mathbf{79.69 \pm 0.13}$	48.38 ± 1.11	14.42 ± 0.30	79.3 ± 4.9
Bagging ($t = 25$)	Natural ($\lambda = 0$)	0.4980 ± 0.0066	78.97 ± 0.25	48.81 ± 0.72	3.56 ± 0.17	963.0 ± 40.3
Boosting ($t = 25, d = 3, \eta = 0.5$)	Natural ($\lambda = 0$)	0.4250 ± 0.0215	76.13 ± 0.56	41.38 ± 1.15	19.38 ± 0.44	84.6 ± 6.8
Boosting ($t = 25, d = 3, \eta = 1.0$)	Natural ($\lambda = 0$)	0.4383 ± 0.0097	74.86 ± 1.47	43.14 ± 1.58	22.11 ± 1.80	54.5 ± 3.0
Unpruned Tree	Balanced ($\lambda = 1$)	0.4797 ± 0.0036	74.42 ± 0.37	50.75 ± 0.12	12.61 ± 0.60	43.5 ± 3.0
Pruned Tree	Balanced ($\lambda = 1$)	$\mathbf{0.5021 \pm 0.0017}$	76.34 ± 0.24	53.78 ± 0.77	17.23 ± 0.74	49.5 ± 3.7
Bagging ($t = 25$)	Balanced ($\lambda = 1$)	0.5015 ± 0.0035	76.77 ± 0.09	53.00 ± 0.23	6.90 ± 0.19	1094.9 ± 15.1
Boosting ($t = 25, d = 3, \eta = 0.5$)	Balanced ($\lambda = 1$)	0.4261 ± 0.0099	67.34 ± 0.39	$\mathbf{58.34 \pm 3.12}$	28.97 ± 1.07	72.1 ± 2.6
Boosting ($t = 25, d = 3, \eta = 1.0$)	Balanced ($\lambda = 1$)	0.4305 ± 0.0184	68.92 ± 1.52	53.67 ± 1.82	27.29 ± 1.11	47.5 ± 4.1



(a) Validation macro-averaged F1-score.



(b) Validation overall accuracy and balanced accuracy.

Fig. 3. Stage Two matched 10,000-row comparison across natural ($\lambda = 0$) and equal-quota balanced ($\lambda = 1.0$) policies across three seeds.

TABLE V
STAGE THREE FULL-GROWTH-POOL PILOT VALIDATION RESULTS ON SEED 42

Configuration	λ	$\bar{f}$	a (%)	$\bar{s}$ (%)	Time (s)
Unpruned Tree ($t = 1$)	0.5	0.5673	77.51	57.32	360.1
Pruned Tree ($t = 1$)	0.5	**0.6003**	**80.67**	61.60	400.1
Bagging ($t = 5$, Full)	0.5	0.5852	78.94	58.77	1725.0
Boosting ($t = 5, d = 3, \eta = 0.5$)	0.5	0.3849	69.88	48.88	105.9
Boosting ($t = 5, d = 3, \eta = 1.0$)	0.5	0.4360	72.14	51.42	99.7
Boosting ($t = 5, d = 8, \eta = 0.5$)	0.5	0.5445	76.67	**61.98**	646.7
Boosting ($t = 5, d = 8, \eta = 1.0$)	0.5	**0.5587**	76.58	61.76	603.6
Unpruned Tree ($t = 1$)	1.0	0.5603	75.64	58.36	320.1
Pruned Tree ($t = 1$)	1.0	0.5915	78.78	**64.43**	360.5
Bagging ($t = 5$, Full)	1.0	0.5789	76.66	60.01	1567.9
Boosting ($t = 5, d = 3, \eta = 0.5$)	1.0	0.3986	62.96	55.71	98.2
Boosting ($t = 5, d = 3, \eta = 1.0$)	1.0	0.3772	60.36	54.96	95.6
Boosting ($t = 5, d = 8, \eta = 0.5$)	1.0	0.5223	73.84	63.50	746.1
Boosting ($t = 5, d = 8, \eta = 1.0$)	1.0	0.5374	73.44	62.08	701.0

broader coverage of *Normal*, *Generic*, and *Exploits* traffic while still allocating at least 6,226 draws to every minority category.

2) **Selection of $d = 8$ and $\eta = 1.0$ for boosting:** Increasing the base-tree depth limit from $d = 3$ to $d = 8$ under $\lambda = 0.5$ increased five-round boosting validation $\bar{f}$ from 0.4360 to 0.5587 at $\eta = 1.0$ and from 0.3849 to 0.5445 at $\eta = 0.5$. Because $d = 8$ with $\eta = 1.0$ achieved the highest validation $\bar{f}$ among the boosting candidates under both $\lambda = 0.5$ and $\lambda = 1.0$, the parameter pair $d = 8$ and $\eta = 1.0$ was selected for the locked primary comparison.

D. Stage Four: Locked Full-Pool Comparison and Ensemble Trajectories

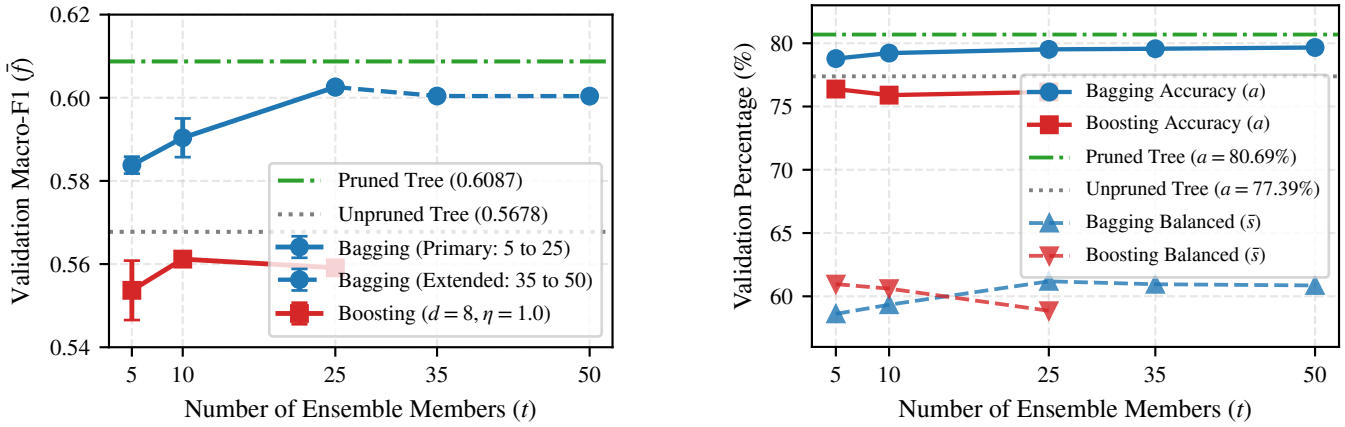
1) Validation Trajectories and Structural Complexity: Table VI and Figure 4 report the validation performance, natural-growth scores, and structural complexity of the locked Stage Four models at $\lambda = 0.5$ across seeds 42 and 43, together with the exploratory extension of bagging to $t = 35$ and $t = 50$ members. All boosting runs fitted all 25 requested members without early termination or learner rejection.

Three patterns were observed in Table VI and Figure 4:

1) **Diminishing returns in bagging beyond 25 members:** Increasing the number of bagging members from one unpruned tree at $\bar{f} = 0.5678$ to five, ten, and 25 members steadily increased validation $\bar{f}$ to 0.5838, 0.5904, and 0.6026, while raising validation overall accuracy from 77.39% to 79.52%. In the exploratory extension from 25 to 35 and 50 members, validation $\bar{f}$ showed no further gain, remaining at 0.6004 at both $t = 35$ and $t = 50$ while overall accuracy changed by $+0.14$ percentage points to 79.66%.

TABLE VI
STAGE FOUR FULL-GROWTH-POOL VALIDATION RESULTS AND STRUCTURAL COMPLEXITY ACROSS TWO SEEDS

Method	Members (t)	Validation $\bar{f}$	Validation a (%)	Validation $\bar{s}$ (%)	Growth $\bar{f}$	Nodes / Leaves / Depth	Size (MB)
Unpruned Tree	1	0.5678 ± 0.0006	77.39 ± 0.18	57.55 ± 0.33	0.7340 ± 0.0015	40,354 / 29,407 / 50.5	0.62
Pruned Tree	1	$\mathbf{0.6087 \pm 0.0119}$	$\mathbf{80.69 \pm 0.02}$	$\mathbf{62.97 \pm 1.94}$	0.6746 ± 0.0047	**3,513 / 1,861** / 42.0	**0.08**
Bagging	5	0.5838 ± 0.0020	78.79 ± 0.22	58.61 ± 0.22	0.7644 ± 0.0024	2.02×10^5 total nodes	3.39
Bagging	10	0.5904 ± 0.0047	79.22 ± 0.09	59.34 ± 0.72	0.7708 ± 0.0002	4.04×10^5 total nodes	6.86
Bagging (Locked)	25	0.6026 ± 0.0003	79.52 ± 0.02	61.18 ± 0.28	$\mathbf{0.7735 \pm 0.0000}$	1.01×10^6 total nodes	17.24
Bagging (Extended)	35	0.6004 ± 0.0001	79.57 ± 0.00	60.95 ± 0.17	0.7738 ± 0.0002	1.41×10^6 total nodes	24.17
Bagging (Extended)	50	0.6004 ± 0.0004	79.66 ± 0.01	60.86 ± 0.10	0.7742 ± 0.0000	2.02×10^6 total nodes	34.56
Boosting	5	0.5537 ± 0.0071	76.38 ± 0.28	60.96 ± 1.13	0.5971 ± 0.0028	$d = 8$ per member	0.88
Boosting	10	0.5612 ± 0.0001	75.90 ± 0.33	60.61 ± 0.09	0.6105 ± 0.0028	$d = 8$ per member	1.75
Boosting (Locked)	25	0.5591 ± 0.0008	76.15 ± 0.55	58.86 ± 0.51	0.6290 ± 0.0026	$d = 8$ per member	4.38



(a) Validation macro-averaged F1-score across ensemble size.

(b) Validation overall accuracy and balanced accuracy.

Fig. 4. Stage Four validation trajectories across ensemble size t at full growth-pool scale ($\lambda = 0.5$) across two seeds.

2) **Early plateau of boosting on validation data:** For depth-eight SAMME boosting, natural-growth $\bar{f}$ increased from 0.5971 at $t = 5$ to 0.6105 at $t = 10$ and 0.6290 at $t = 25$. On the validation set, however, $\bar{f}$ peaked at $\bar{f} = 0.5612$ for $t = 10$ and measured 0.5591 at $t = 25$, while validation balanced accuracy decreased from 60.96% at $t = 5$ to 58.86% at $t = 25$.
3) **Structural reduction from reduced-error pruning:** The unpruned full-pool tree averaged 40,354 internal decision nodes, 29,407 leaves, and a depth of 50.5, occupying 0.62 MB on disk. Reduced-error pruning on the 41,225-row pruning set removed 91.3% of internal nodes and 93.7% of leaves, yielding a mean of 3,513 internal nodes and 1,861 leaves with a disk size of 0.08 MB while increasing validation $\bar{f}$ from 0.5678 to 0.6087.

2) Primary Generalisation Set Evaluation: Table VII reports the primary evaluation of the eight locked models on the reserved 51,531-row generalisation set across seeds 42 and 43.

The generalisation rankings in Table VII matched the validation rankings in both seeds:

1) **Pruned single tree:** The pruned tree achieved the highest score on all three primary generalisation metrics with a mean $\bar{f}$ of 0.5993 ± 0.0028, an overall accuracy of $80.73\% \pm 0.17\%$, and a balanced accuracy of $63.02\% \pm 0.13\%$. Relative to the unpruned tree, post-pruning improved $\bar{f}$ by $+0.0448$, overall accuracy by $+3.29$ percentage points, and balanced accuracy by $+6.37$ percentage points, while recording the lowest batch prediction time of 0.48 s across the 51,531 rows.
2) **Bagging with 25 members:** Bagging ranked second with a mean $\bar{f}$ of 0.5747 ± 0.0006, outperforming the unpruned single tree in both seeds by a mean of $+0.0202$ in $\bar{f}$, $+2.11$ percentage points in overall accuracy at 79.55%, and $+1.87$ percentage points in balanced accuracy at 58.52%. Relative to the pruned single tree, 25-member bagging trailed by 0.0246 in $\bar{f}$ and 4.50 percentage points in balanced accuracy while requiring 6.61 s for batch prediction.
3) **Unpruned single tree:** The unpruned tree ranked third in $\bar{f}$ at 0.5545 ± 0.0055 and third in overall accuracy at 77.44%.
4) **Boosting with 25 members:** Depth-eight SAMME boosting ranked fourth in $\bar{f}$ at 0.5408 ± 0.0074 and fourth in overall accuracy at 76.19%, while achieving a higher balanced accuracy at 57.26% than the unpruned single tree at 56.65%.

TABLE VII
PRIMARY EVALUATION ON THE RESERVED GENERALISATION SET ACROSS SEEDS 42 AND 43

Approach	Members (t)	Seed	Generalisation $\bar{f}$	Overall Accuracy a (%)	Balanced Accuracy $\bar{s}$ (%)	Prediction Time (s)
Unpruned Tree	1	42	0.5584	77.50	56.94	1.02
Unpruned Tree	1	43	0.5506	77.38	56.35	0.54
Unpruned Tree	**1**	**Mean**	$\mathbf{0.5545 \pm 0.0055}$	$\mathbf{77.44 \pm 0.09}$	$\mathbf{56.65 \pm 0.41}$	$\mathbf{0.78 \pm 0.34}$
Pruned Tree	1	42	0.6013	80.85	62.93	0.48
Pruned Tree	1	43	0.5973	80.61	63.12	0.48
Pruned Tree	**1**	**Mean**	$\mathbf{0.5993 \pm 0.0028}$	$\mathbf{80.73 \pm 0.17}$	$\mathbf{63.02 \pm 0.13}$	$\mathbf{0.48 \pm 0.00}$
Bagging	25	42	0.5752	79.65	58.63	6.65
Bagging	25	43	0.5742	79.45	58.42	6.58
Bagging	**25**	**Mean**	$\mathbf{0.5747 \pm 0.0006}$	$\mathbf{79.55 \pm 0.14}$	$\mathbf{58.52 \pm 0.15}$	$\mathbf{6.61 \pm 0.05}$
Boosting ($d = 8, \eta = 1.0$)	25	42	0.5460	75.96	57.97	4.57
Boosting ($d = 8, \eta = 1.0$)	25	43	0.5355	76.42	56.54	5.24
Boosting ($d = 8, \eta = 1.0$)	**25**	**Mean**	$\mathbf{0.5408 \pm 0.0074}$	$\mathbf{76.19 \pm 0.33}$	$\mathbf{57.26 \pm 1.01}$	$\mathbf{4.91 \pm 0.48}$

E. Class-Specific Error Analysis and Model Comparison

Table VIII reports the mean per-class precision p_j, recall s_j, and F1-score f_j across all ten traffic categories on the generalisation set. Figure 5 displays the per-class F1-scores, recalls, and precisions, while Figure 6 presents the row-normalised recall confusion matrices for the pruned tree and 25-member bagging.

1) Comparison of Pruned Trees and Unpruned Bagging: Comparing the unpruned standalone tree directly with the corresponding pruned tree in Table VIII isolates the effect of reduced-error post-pruning on the exact same initial tree fit. Post-pruning increased *Backdoor* precision p_3 from 0.0428 to 0.1027 and raised f_3 from 0.0693 to 0.1252. On *Analysis*, post-pruning increased precision p_6 from 0.0541 to 0.1438 and raised f_6 from 0.0782 to 0.1315. On *DoS*, post-pruning increased recall s_4 from 0.3578 to 0.5740 and raised f_4 from 0.3385 to 0.4364. On *Worms*, post-pruning increased recall s_8 from 0.5735 to 0.7941 and raised f_8 from 0.5571 to 0.6393.

When 25 unpruned trees trained under the same $\lambda = 0.5$ balanced distribution were combined via bagging, the ensemble achieved the highest F1-score on four categories in Table VIII: *Normal* with $f_1 = 0.9249$, *Fuzzers* with $f_7 = 0.6444$, *Shellcode* with $f_9 = 0.6458$, and *Generic* with $f_{10} = 0.9885$, while matching the pruned tree on *Exploits* with $f_5 = 0.6915$. However, on the categories where the $\lambda = 0.5$ training quotas heavily oversampled rare classes that overlap with *DoS* and *Exploits*, unpruned bagging exhibited the same false-positive pattern as the unpruned standalone tree:

- As shown in Figure 6b, 25-member bagging misclassified 27% of true *DoS* records and 12% of true *Exploits* records as *Backdoor*, and misclassified an additional 12% of true *DoS* records and 5% of true *Exploits* records as *Analysis*.
- As indicated in Table VIII, the precision of 25-member bagging was 0.0398 on *Backdoor* and 0.0537 on *Analysis*, while *DoS* recall was 0.3466.
- In the pruned tree confusion matrix in Figure 6a, false *DoS*-to-*Backdoor* predictions measured only 6% and false *DoS*-to-*Analysis* predictions measured 3%, while true *DoS* recall reached 57%.

The observed contrast between unpruned bagging and the pruned tree is consistent with the difference between averaging unpruned trees trained on an oversampled distribution and post-pruning a balanced tree against a natural-distribution pruning set P. While individual internal split paths of the 50 bagging trees were not audited, pruning directly evaluated subtree error counts on the natural class proportions of P, collapsing subtrees that generated more false-positive errors on abundant classes than true-positive detections on oversampled classes.

2) Multiclass Adaptive Boosting Performance: Although depth-eight SAMME boosting improved substantially over the depth-three boosting models tested in Stages Two and Three, the 25-round generalisation macro-averaged F1-score of boosting reached $\bar{f} = 0.5408$, trailing both 25-member bagging at 0.5747 and the unpruned single tree at 0.5545. In Table VIII, boosting achieved the highest recall on *Analysis* with $s_6 = 0.1673$ and on *Shellcode* with $s_9 = 0.7815$. However, boosting recorded lower recall on *Normal* with $s_1 = 0.8670$ versus 0.9349 for bagging, lower F1-score on *Fuzzers* with $f_7 = 0.5360$ versus 0.6444 for bagging, and lower precision on *Shellcode* with $p_9 = 0.4380$ versus 0.5865 for bagging and on *Reconnaissance* with $p_2 = 0.7781$ versus 0.9048 for the pruned tree. Lower precision on *Shellcode* and *Reconnaissance* is consistent with the sensitivity of exponential reweighting to label noise and overlapping class boundaries [3], documented in Stage One through the 1,508 conflicting feature vectors at 50,000 rows, although per-instance weight trajectories across the 25 boosting rounds were not separately logged.

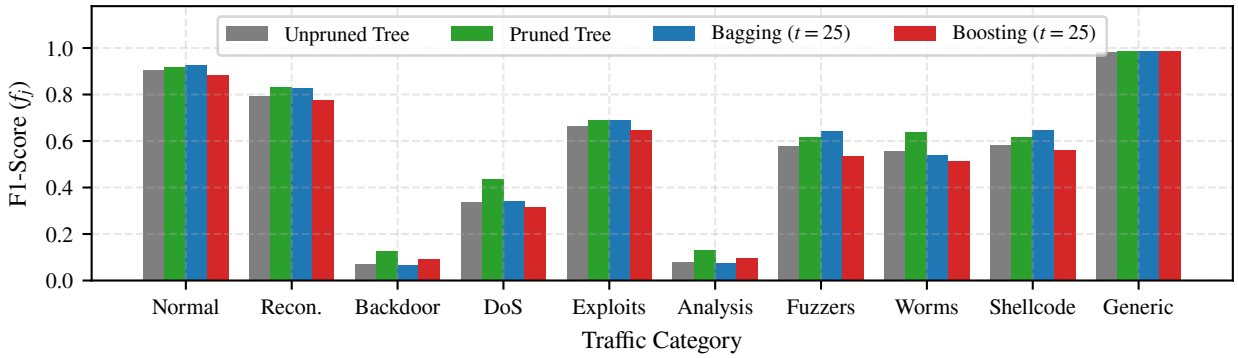
F. Methodological Limitations

Four methodological constraints govern the interpretation of the experimental findings:

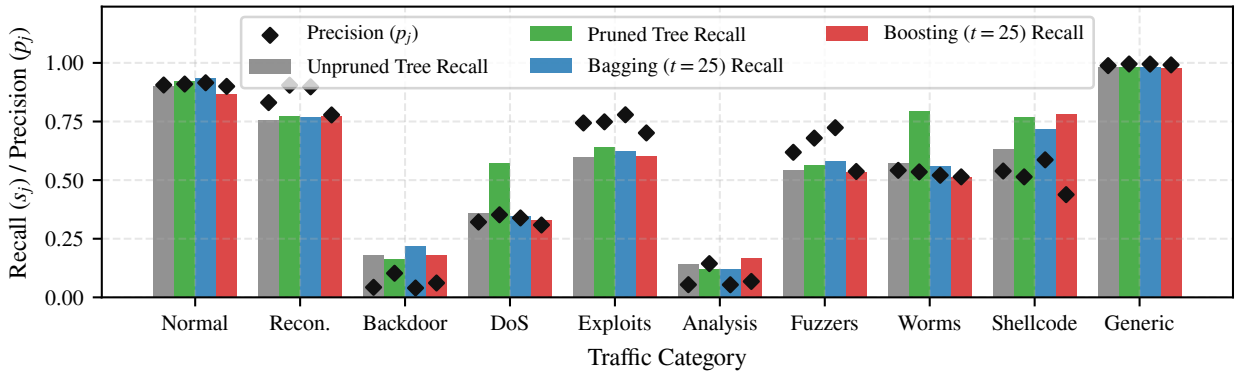
- **Row-based random splitting and repeated feature vectors:** Although the four-way partition was stratified by class, rows were assigned randomly rather than partitioned by capture time window or internet protocol (IP) subnet. Because repeated identical 41-attribute vectors occur in the dataset and were retained to preserve natural flow frequencies during induction and pruning, identical predictor vectors could appear in both the growth and generalisation partitions. Consequently, the reported metrics measure generalisation across held-out rows of the same traffic

TABLE VIII
PER-CLASS PRECISION (p_j), RECALL (s_j), AND F1-SCORE (f_j) ON THE RESERVED GENERALISATION SET ACROSS TWO SEEDS

			Unpruned Tree			**Pruned Tree**			**Bagging** ($t = 25$)			**Boosting** ($t = 25$)		
Index (j)	**Category** (c_j)	**Support**	p_j	s_j	f_j	p_j	s_j	f_j	p_j	s_j	f_j	p_j	s_j	f_j
1	Normal	18,600	0.9057	0.9010	0.9034	0.9098	0.9234	0.9165	**0.9152**	**0.9349**	**0.9249**	0.8998	0.8670	0.8831
2	Reconnaissance	2,797	0.8307	0.7560	0.7916	**0.9048**	**0.7735**	**0.8340**	0.8979	0.7690	0.8285	0.7781	0.7708	0.7744
3	Backdoor	465	0.0428	0.1817	0.0693	**0.1027**	0.1613	**0.1252**	0.0398	**0.2183**	0.0673	0.0612	0.1806	0.0914
4	DoS	3,270	0.3212	0.3578	0.3385	**0.3521**	**0.5740**	**0.4364**	0.3387	0.3466	0.3426	0.3086	0.3275	0.3177
5	Exploits	8,905	0.7438	0.5970	0.6624	0.7486	**0.6424**	**0.6915**	**0.7788**	0.6218	**0.6915**	0.7013	0.6026	0.6482
6	Analysis	535	0.0541	0.1411	0.0782	**0.1438**	0.1215	**0.1315**	0.0537	0.1215	0.0745	0.0678	**0.1673**	0.0965
7	Fuzzers	4,849	0.6187	0.5415	0.5775	0.6793	0.5626	0.6154	**0.7236**	**0.5807**	**0.6444**	0.5369	0.5352	0.5360
8	Worms	34	0.5417	0.5735	0.5571	0.5351	**0.7941**	**0.6393**	0.5206	0.5588	0.5390	0.5139	0.5147	0.5142
9	Shellcode	302	0.5388	0.6341	0.5824	0.5138	0.7699	0.6162	**0.5865**	0.7185	**0.6458**	0.4380	**0.7815**	0.5614
10	Generic	11,774	0.9884	0.9808	0.9846	**0.9953**	0.9798	0.9875	0.9950	**0.9820**	**0.9885**	0.9915	0.9785	0.9850
Macro Average		**51,531**	0.5586	0.5665	0.5545	**0.5885**	**0.6302**	**0.5993**	0.5850	0.5852	0.5747	0.5297	0.5726	0.5408



(a) Per-class F1-score (f_j) across the ten traffic categories.

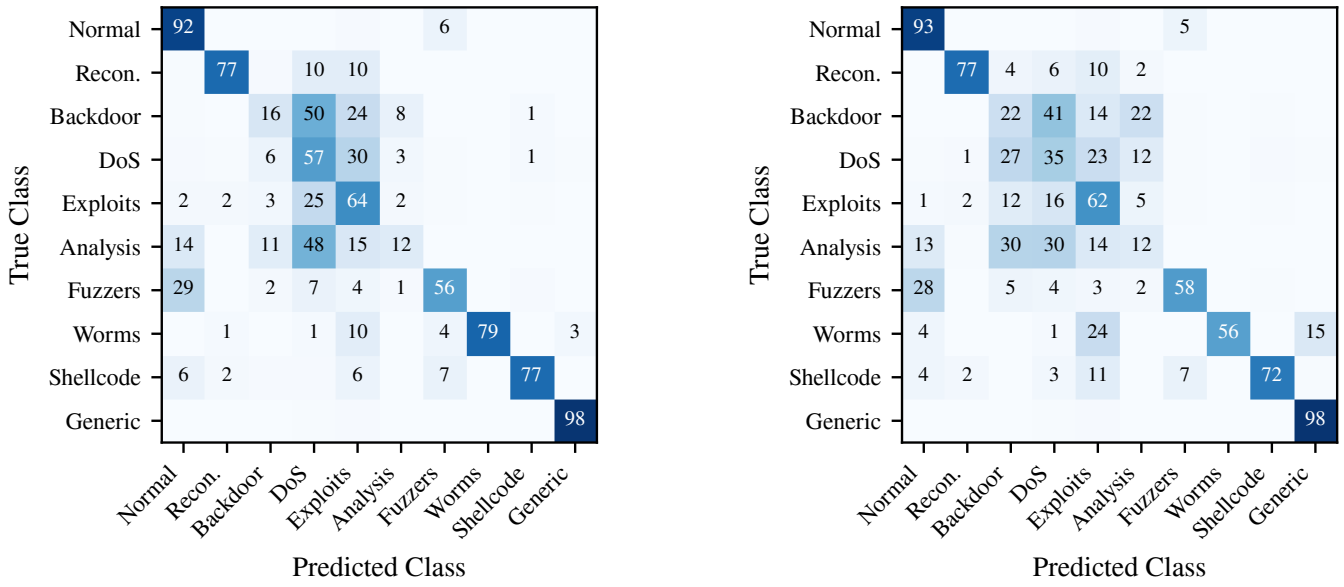


(b) Per-class recall (s_j, vertical bars) and precision (p_j, black diamonds).

Fig. 5. Per-class generalisation performance across the ten traffic categories across two seeds.

collection rather than deployment to an unseen network environment.

- **Minority support and oversampling:** The entire 257,673-row dataset contained 174 *Worms* records, with 84 in the growth pool, 28 in the validation pool, and 34 in the generalisation pool. Oversampling 84 records into 6,226 draws increased the weight of *Worms* during split selection, but did not introduce new attack variations. On 34 generalisation instances, a single prediction change altered *Worms* recall s_8 by $1/34 \approx 2.94$ percentage points.
- **Unpruned ensemble members and unweighted pruning criterion:** In the locked comparison, reduced-error post-pruning was applied only to the standalone tree, while bagging aggregated unpruned trees. In addition, the pruning criterion in Equation (11) minimised unweighted classification errors rather than directly maximising macro-averaged F1-score $\bar{f}$.
- **Number of random seeds:** Because full-pool induction on 123,683 rows was computationally intensive, the locked comparison used two random seeds, 42 and 43, evaluated on a single shared generalisation partition. Reported standard deviations therefore describe variation across resampling seeds rather than confidence intervals across independent datasets.



(a) Pruned single decision tree.

(b) Bagging with 25 unpruned members.

Fig. 6. Row-normalised generalisation confusion matrices showing class recall percentages across seeds 42 and 43.

VI. Conclusion

The present study investigated how the predictive performance of a standalone decision tree compared with decision-tree ensemble methods for ten-class network intrusion detection on a 257,673-row subset of the University of New South Wales network-based 2015 (UNSW-NB15) dataset [4]. An extended iterative dichotomiser three (ID3) decision-tree learner, bottom-up reduced-error post-pruning, bootstrap aggregation, known as bagging, and stagewise additive modelling using a multi-class exponential loss function (SAMME) were implemented within a unified software framework so that the base classifier family and 41-attribute representation remained constant across all models. Using a fixed, stratified four-way split that separated model growth with 123,683 rows, reduced-error pruning with 41,225 rows, development validation with 41,234 rows, and final generalisation testing with 51,531 rows, four sequential experimental stages were executed to answer the four research questions of the study.

First, regarding sample-size scaling and class scarcity, the Stage One learning-curve experiments across 2,000 to 50,000 rows showed that increasing the volume of original training data produced larger gains in minority-attack recognition than applying ensemble methods to small subsamples. Under natural class proportions, rare categories such as *Worms*, representing 0.07% of traffic, had too few distinct instances in small samples for bootstrap resampling to overcome. Grouping the training samples by exact 41-attribute vectors further demonstrated that the decrease in unrestricted training accuracy from 99.00% at 2,000 rows to 93.26% at 50,000 rows matched the exact deterministic ceiling imposed by identical predictor vectors carrying conflicting attack labels.

Second, regarding the interaction between structural regularisation and class balancing, Stage One and Stage Two showed that unweighted reduced-error pruning behaved differently depending on the training class distribution. When trees were induced on naturally skewed data, post-pruning consistently increased overall accuracy a while reducing macro-averaged F1-score $\bar{f}$ by collapsing sparse minority-class subtrees. Conversely, when minority classes were oversampled during tree induction and the induced tree was post-pruned against a natural-proportion pruning set, class balancing and post-pruning acted complementarily. In the Stage Three full-pool pilot, hybrid class balancing at $\lambda = 0.5$ outperformed pure equal-quota balancing at $\lambda = 1.0$ across every model family in both validation $\bar{f}$ and overall accuracy a.

Third, regarding ensemble behaviour, 25-member bagging consistently improved over unpruned single trees across all tested sample sizes and class policies, raising generalisation $\bar{f}$ from 0.5545 to 0.5747 and overall accuracy from 77.44% to 79.55%. Extending bagging from 25 to 35 and 50 members on the validation set produced no further gain in $\bar{f}$, measuring 0.6026 at 25 members versus 0.6004 at 35 and 50 members. Sequential SAMME boosting benefited substantially from increasing the base-tree depth limit from $d = 3$ to $d = 8$ at learning rate $\eta = 1.0$, but lagged behind both bagging and the unpruned tree on the generalisation set with $\bar{f} = 0.5408$, exhibiting lower precision on *Shellcode* and *Reconnaissance* and lower recall on *Normal* traffic.

Fourth, in the locked full-pool generalisation comparison under the shared $\lambda = 0.5$ hybrid balancing policy, the standalone reduced-error-pruned decision tree achieved the strongest overall performance across all three primary metrics: a mean macro-averaged F1-score of 0.5993, a balanced accuracy of 63.02%, and an overall accuracy of 80.73%, while reducing mean leaf count by 93.7% to 1,861 leaves occupying 0.08 megabytes (MB) and completing batch inference 13.8 times

faster than 25-member bagging. While 25-member bagging achieved the highest F1-score on *Normal*, *Fuzzers*, *Shellcode*, and *Generic*, unpruned bagging generated substantially more false *Backdoor* and *Analysis* predictions on true *DoS* and *Exploits* traffic than the pruned standalone tree.

Four directions for future work arise directly from the findings and limitations of the present study:

1) **Bagging of post-pruned balanced trees:** Because bagging improved F1-score on *Normal*, *Fuzzers*, *Shellcode*, and *Generic* while reduced-error post-pruning suppressed false alarms on oversampled *Backdoor* and *Analysis* leaves, applying reduced-error pruning to each individual balanced bagging member is a natural extension.
2) **Macro-averaged F1-score pruning criteria:** Replacing the unweighted error count in reduced-error pruning with a class-weighted error criterion or direct validation $\bar{f}$ optimisation would align the post-pruning objective with balanced multiclass detection.
3) **Deduplicated and temporal-split evaluation:** Future experiments on UNSW-NB15 should evaluate deduplicated flow sets and partition records by capture time window or internet protocol (IP) subnet to measure generalisation across distinct network sessions.
4) **Random attribute subspace selection:** Extending the bagging implementation to sample a random subset of descriptive attributes at each split node would decorrelate root splits dominated by high-cardinality `proto` and `service` attributes.

References

[1] L. Breiman, "Bagging predictors," *Machine Learning*, vol. 24, no. 2, pp. 123–140, 1996.

[2] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.

[3] J. D. Kelleher, B. Mac Namee, and A. D'Arcy, *Fundamentals of Machine Learning for Predictive Data Analytics: Algorithms, Worked Examples, and Case Studies*, 2nd ed. Cambridge, MA: MIT Press, 2020.

[4] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proceedings of the 2015 Military Communications and Information Systems Conference*. Institute of Electrical and Electronics Engineers, 2015, pp. 1–6.

[5] J. R. Quinlan, "Induction of decision trees," *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986.

[6] ——, *C4.5: Programs for Machine Learning*. San Mateo, CA: Morgan Kaufmann, 1993.

[7] C. E. Shannon, "A mathematical theory of communication," *The Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.

[8] J. Zhu, H. Zou, S. Rosset, and T. Hastie, "Multi-class AdaBoost," *Statistics and Its Interface*, vol. 2, no. 3, pp. 349–360, 2009.

Appendix A
List of Acronyms

Table IX lists all acronyms used in the report in alphabetical order.

TABLE IX
Alphabetical List of Acronyms

Acronym	Definition
FTP	file transfer protocol
ID3	iterative dichotomiser three
IP	internet protocol
MB	megabyte
NIDS	network intrusion detection system
SAMME	stagewise additive modelling using a multi-class exponential loss function
SHA-256	secure hash algorithm 256-bit
TCP	transmission control protocol
UNSW-NB15	University of New South Wales network-based 2015

Appendix B
List of Symbols

Table X lists all mathematical symbols used in the report in alphabetical order, grouped into Latin symbols followed by Greek symbols.

TABLE X
ALPHABETICAL LIST OF MATHEMATICAL SYMBOLS

Symbol	Definition	Symbol	Definition
a	overall classification accuracy	P_v	subset of pruning dataset P reaching internal node v
b	descriptive attribute index, $b \in \{1, \ldots, m\}$	p_j	precision for target class c_j
C	set of target classes, $C = \{c_1, \ldots, c_k\}$	q	predicted class index in matrix $\mathbf{M}$, $q \in \{1, \ldots, k\}$
c_j	target class label j, $c_j \in C$	r	ensemble member or boosting round index, $r \in \{1, \ldots, t\}$
$\mathbf{c}_1$	running class-count vector for the left threshold branch	s_j	recall for target class c_j
$\mathbf{c}_2$	running class-count vector for the right threshold branch	$\bar{s}$	balanced accuracy across all k classes
$\mathbf{c}_\tau$	class-count vector at distinct numerical value v_τ	t	total number of base learners in an ensemble
D	dataset of labelled instances	u	number of distinct ordered values of a numerical attribute
D_1	left child subset of a binary threshold split ($x_{i,b} < \theta$)	v	internal node in a decision tree
D_2	right child subset of a binary threshold split ($x_{i,b} \geq \theta$)	v_τ	distinct ordered value τ of a numerical attribute
D_f	fitting dataset used to induce a decision tree	W_r	set of misclassified training instance indices at round r
D_l	subset of dataset D where attribute b equals level l	$\mathbf{w}_r$	observation weight vector at boosting round r
D_r	resampled training dataset for ensemble member r	$w_{r,i}$	observation weight of instance i at boosting round r
D_v	subset of fitting dataset D_f reaching internal node v	$\mathbf{x}_i$	descriptive attribute vector for instance i
d	maximum tree depth limit	$x_{i,b}$	value of descriptive attribute b for instance i
e_v	unweighted error count of the subtree at node v on P_v	y_i	true target class label of instance i
f_j	F1-score for target class c_j	y_v^*	modal training class label at internal node v
$\bar{f}$	macro-averaged F1-score across all k classes	z_r	weight normalisation divisor at boosting round r
$h(D)$	Shannon entropy of dataset D in bits	α_r	voting confidence weight of ensemble member r
i	instance index, $i \in \{1, \ldots, n\}$	β_j	natural proportion of class c_j in the growth pool
j	target class index, $j \in \{1, \ldots, k\}$	γ_j	hybrid class-sampling proportion for class c_j
k	total number of target classes	$\delta(\cdot)$	indicator function returning 1 if true and 0 otherwise
L_b	discrete domain of levels for categorical attribute b	ϵ_r	weighted training error of base learner r
l	categorical level in domain L_b	η	boosting learning rate
$\ell_{r,i}$	natural logarithm of weight $w_{r,i}$ at round r	$\zeta(D)$	Gini impurity of dataset D
$\hat{\ell}_r$	maximum misclassified log-weight at boosting round r	θ	binary split threshold for a numerical attribute
$\mathbf{M}$	$k \times k$ confusion matrix	$\iota(b, D)$	information gain of splitting dataset D on attribute b
m	total number of descriptive attributes	$\kappa(b, D)$	split information of partitioning dataset D on attribute b
m_{jq}	count of true class c_j predicted as class c_q in $\mathbf{M}$	λ	hybrid class-balancing interpolation weight
n	total number of instances in a dataset or draw budget	μ	Pearson linear correlation coefficient
n_j	integer training draw quota allocated to class c_j	$\nu(D, c_j)$	empirical proportion of class c_j in dataset D
o_v	unweighted error count of a modal leaf at node v on P_v	$\xi(b, D)$	expected remaining entropy after splitting D on attribute b
P	pruning dataset used for reduced-error post-pruning	π_r	classifier function induced for ensemble member r
		$\hat{\pi}$	combined ensemble classifier function
		ρ	Spearman rank correlation coefficient
		τ	index over distinct ordered values, $\tau \in \{1, \ldots, u\}$
		$\psi(b, D)$	gain ratio of splitting dataset D on attribute b
		ω	sub-bagging subsample size drawn without replacement